Problem 3: Chunked Flash Linear Attention

GStar Bootcamp

Abstract

Build chunk-boundary states and parallel chunk outputs for linear attention.

Starter file: `problem_3.py`. **Checker:** `python autograder.py -p3`. **Reading path:** Overview, P1, P2. Next: P4.

Mathematical background: group tokens without losing causality

The token loop is convenient for decoding, but a long prompt offers many tokens to process together. Define the summary of a chunk by

$$\Delta \mathbf{S}^{(c)} = \sum_{r=1}^{C} \boldsymbol{k}_r \boldsymbol{v}_r^\top = \mathbf{K}^\top \mathbf{V}.$$

Addition is associative. Grouping token writes into chunk summaries therefore preserves the boundary state, with exact arithmetic independent of grouping. A prefix scan of these summaries could produce all boundaries; the supplied state kernel uses a simple sequential pass over chunks.

A boundary summary alone is insufficient for outputs *inside* its chunk. At local row r, split visible history into the entry memory and the first r local writes:

$$\boldsymbol{o}_r = \mathbf{S}_{\text{in}}^\top \boldsymbol{q}_r + \sum_{j \le r} (\boldsymbol{q}_r^\top \boldsymbol{k}_j) \boldsymbol{v}_j.$$

The first term is a read of old memory; the second is a causal local attention computation. Stacking rows turns both into matrix products. This is why the chunk algorithm has a boundary pass followed by independent output programs, rather than a single full-sequence score matrix [1].

Chunking changes the grouping of operations, not which past tokens matter. For fixed C, its principal arithmetic scales as $O(TD_kD_v + TC(D_k + D_v))$, suppressing batch/head factors. Increasing C trades fewer boundaries for larger local score products. Floating-point rounding can change with grouping even though the underlying operator is identical.

Before coding. For a query in the middle of a chunk, separate the tokens represented by $\mathbf{S}_{\text{in}}$ from the tokens represented by its local score row. The sets must be disjoint and together contain exactly the visible prefix.

Two phases, one recurrence

Use the formulas derived in Problem 1. First build the state entering every chunk; then launch independent output programs for those chunks. For $N = \lceil T/C \rceil$, allocate

$$\texttt{States} \in \mathbb{R}^{B \times H \times (N+1) \times D_k \times D_v}.$$

Entry 0 is the initial state. Entry $c + 1$ is the state after chunk c.

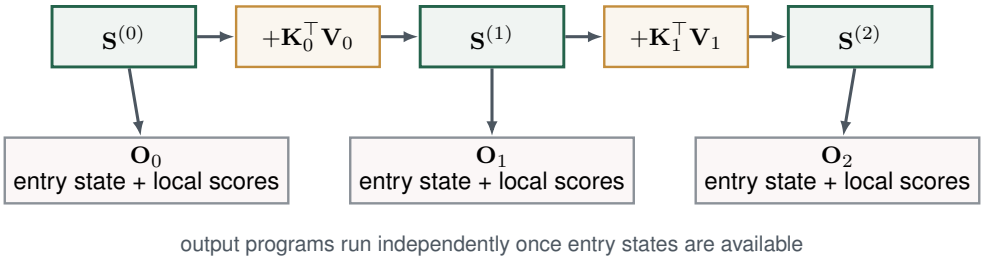


Figure 1. A sequential pass over chunk boundaries provides states. Output work inside separate chunks can then run in parallel. The diagram shows three output chunks and two state transitions.

State-kernel grid

Assign one program to a $(\text{key tile}, \text{value tile}, bH + h)$ triple. It loops over chunks and updates its (B_K, B_V) state tile with $\mathbf{K}_{\text{tile}}^\top \mathbf{V}_{\text{tile}}$. With $B_K = B_V = 64$, the grid is

$$(D_k/B_K,\ D_v/B_V,\ BH).$$

Output-kernel grid

Assign one program to a $(\text{chunk}, \text{value tile}, bH + h)$ triple. It loops over key tiles, accumulating the state read and the $C \times C$ scores. Its grid is $(N, D_v/B_V, BH)$.

Problem 3: step-by-step kernel construction

1. **Build chunk states (45 points).** Load the initial state tile. For each chunk, load key and value tiles, compute their transposed matrix product, add it to the carried state, and store state entry $c + 1$.
2. **Compute the entry-state contribution (25 points).** For an output chunk, initialize a (C, B_V) FP32 accumulator. Sum $\mathbf{Q}_{\text{tile}} \mathbf{S}_{\text{tile}}^{(c)}$ over key tiles.
3. **Compute local causal output (40 points).** Sum $\mathbf{Q}_{\text{tile}} \mathbf{K}_{\text{tile}}^\top$ into a (C, C) score tile. Zero invalid upper-triangle entries, multiply the resulting tile by the values, and combine it with the entry-state term. Apply scale once.
4. **Explain parallelism (15 points).** Identify the sequential dependency in the state pass and the independent dimensions of the output pass.

```
# _chunk_output_forward: tile loads and final store are provided.
inter = tl.zeros([CHUNK, BLOCK_V], tl.float32)
scores = tl.zeros([CHUNK, CHUNK], tl.float32)
for key_block in range(HEAD_DIM // BLOCK_K):
    # q, k, state are loaded with their documented shapes.
    # TODO P3.A: accumulate the entry-state read into inter.
    # TODO P3.B: accumulate query-key products into scores.
# TODO P3.C: zero future entries using local chunk indices.
# TODO P3.D: multiply scores by v and combine with inter.
```

Mask values are zero

These scores weight values directly. Invalid entries must contribute zero. Softmax attention often uses $-\infty$ before exponentiation; these linear-attention score tiles do not apply softmax. An invalid linear weight of $-\infty$ would corrupt the matrix product.

Memory accounting

The recurrent state itself has fixed size per head. This chunk implementation additionally saves chunk-boundary states for parallel outputs. State-buffer storage is

$$O(BH(N+1)D_kD_v).$$

If score tiles are materialized, their total storage is $O(BHTC)$, which grows linearly with T for fixed C. A full $T \times T$ attention matrix is prohibited in the chunked submissions.

Verification

Check outputs and final state against Problems 1 and 2. In addition, compare each stored chunk state against the sequential recurrence at the matching boundary. Test $T = 63, 64, 65, 129, 200$, two value tiles, and two key tiles. Incorrect state timing usually appears immediately in the second chunk.

Problem 3: chunk boundaries, reductions, and partial chunks

State timing for a 65-token sequence

With $C = 64$, pad to $\widetilde{T} = 128$ and allocate three state entries:

$$\mathbf{S}^{(0)} = \mathbf{S}_0, \qquad \mathbf{S}^{(1)} = \mathbf{S}_{64}, \qquad \mathbf{S}^{(2)} = \mathbf{S}_{65}.$$

The second chunk contains one real token and 63 neutral padded tokens. Its output kernel reads $\mathbf{S}^{(1)}$, not $\mathbf{S}^{(2)}$. The final returned memory is $\mathbf{S}^{(2)}$. If the second output chunk reads its exit state, it counts its own writes twice through the inter-chunk and local terms.

Accumulate all key tiles before masking

Let the key dimension be split into tiles $\mathcal{D}_m$. The output program computes

$$\mathbf{H} = \sum_m \mathbf{Q}_{:,\mathcal{D}_m} \mathbf{S}^{(c)}_{\mathcal{D}_m,:}, \tag{3.1}$$

$$\mathbf{Z} = \sum_m \mathbf{Q}_{:,\mathcal{D}_m} \mathbf{K}^\top_{:,\mathcal{D}_m}, \tag{3.2}$$

$$\mathbf{O} = \big[\mathbf{H} + \mathrm{tril}(\mathbf{Z})\mathbf{V}\big]. \tag{3.3}$$

The score tile is a sum across key coordinates, not across value coordinates. The local value multiplication follows completion of that sum. Your output tile's value width is B_V; $\mathbf{Z}$ remains $C \times C$ regardless of D_v.

Different state passes have different ownership

In plain linear attention, $\mathbf{K}^\top\mathbf{V}$ is additive and separable across both state axes. A state-pass program may own one key/value tile and scan all chunks independently. In delta-family chunking, actual pseudo-values depend on *all* key rows of the entry memory. Later state kernels therefore own a value tile and perform a complete key reduction before updating state tiles. Keep this distinction when reusing the scheduling idea.

Arithmetic work and storage

For one batch/head, the state pass performs matrix products costing $O(TD_kD_v)$. Output inter-chunk reads cost $O(TD_kD_v)$, local query–key work costs $O(TCD_k)$, and local value products cost $O(TCD_v)$. For fixed C, D_k, D_v, each quantity grows linearly in T.

Stored boundaries cost $(N+1)D_kD_v$ elements. A materialized local score buffer costs $\widetilde{T}C$ elements; the P3 output kernel can instead hold one score tile on chip. These are different storage decisions even though both avoid a full $T \times T$ matrix.

Reasoning questions for your submission

Explain why output chunks can run independently after the boundary pass; why the boundary pass itself scans chunks; and why $D_k = 128$ requires two accumulated score contributions. Describe which program owns `out[b,h,t,a]` and show that no other output program writes that address.

References

[1] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*. 2020.
https://arxiv.org/abs/2006.16236